Columbia University

Multimodal Encoding AI Framework for Retail Forecasting

Advisors:
Professor Ali Hirsa
John Bible
Bat-Orgil Batjargal
Similoluwa Gloria Adelore
Miao Wang

Submitted by:
Frank Shi (ps3391)
Meijia Zhu (mz3060)

September 2025

Contents

1 Background & Motivation

Understanding and forecasting customer purchasing behavior is of central importance in modern retail analytics. Accurate prediction of user baskets offers substantial commercial value: It enables retailers to improve recommendation systems, optimize inventory management, and improve the overall shopping experience. However, the fundamental question remains: *How can we accurately forecast a customer's basket - both during the current trip and in future visits?*

This problem is inherently challenging. Customer baskets are complex and sequential in nature, consisting of various items that may reflect multiple shopping intentions. Moreover, these baskets evolve over time, shaped by dynamic user preferences and contextual factors. Building effective models on such data requires the ability to capture both static item relationships and temporal dependencies across shopping sessions.

Equally important is the challenge of model design itself. Developing architectures that balance predictive accuracy with generalization capability is not trivial. Machine learning models must be carefully tuned, both in terms of architectural choices and hyperparameter settings, to ensure that performance gains on training data translate effectively to unseen scenarios.

Our work is motivated by these challenges. We aim to develop machine learning models that can learn from historical basket data and provide accurate and actionable predictions for both within-basket and next-basket forecasting tasks. Although prior studies have shown encouraging results, the problem remains far from solved and requires further methodological advances.

To address these challenges, we employ and extend three representative graph-based architectures. The first is the Multi-Intent Translation Graph Neural Network (MITGNN) [3], which is designed to capture diverse shopping intents within a basket. We further integrate sequential modeling through a graph-based recurrent neural network (GRNN) [6], allowing temporal dynamics to be incorporated into predictions. Finally, we explore the Temporal Graph Network (TGN) [5], which explicitly models the evolution of user-item interactions over time. Together, these models provide complementary perspectives on the structure, sequence, and temporal change in customer purchasing behavior, forming the methodological foundation of our study.

2 Prior Work & Introduction & Contribution

2.1 Prior work summarization

This project builds upon a prior team's efforts to explore graph-based models for grocery basket prediction. Their work focused on extending the Multi-Intent Translation Graph Neural Network (MITGNN) and its variants to both within-basket and next-basket forecasting tasks. In doing so, they made three primary contributions.

First, they attempted to adapt models originally designed for within-basket prediction (such as MITGNN, MITGNN+GRNN, and MITGNN+TGN) to the next-basket setting. During this process, they identified methodological limitations in the initial implementations and highlighted the importance of proper data transformation and index mapping when repurposing within-basket architectures for future-basket forecasting.

Second, they enhanced item embeddings by moving beyond simple index-based representations. Specifically, they introduced two additional embedding modalities: (1) textual embeddings derived from item descriptions using pre-trained BERT, and (2) price embeddings constructed via sinusoidal encoding of unit prices. These enhancements aimed to provide richer semantic and numerical information for model training.

Third, they developed a client-oriented predictive interface that bridged research outputs with practical applications. While traditional evaluation relied on academic metrics such as recall, precision, and NDCG, their interface allowed users to input specific IDs and receive direct recommendations for within-basket or next-basket predictions. This effort demonstrated the potential of deploying graph-based recommender systems in real-world commercial contexts.

Together, these contributions established a strong foundation for basket recommendation research, while also revealing key challenges in model adaptation, multimodal representation, and practical usability that motivate our continued investigation.

2.2 Models Introduction

2.2.1 Graph Neural Networks (GNNs)

GNNs learn node representations by passing messages along edges in a graph. For basket prediction, we model users, items, and optionally baskets as nodes;

edges encode interactions such as purchases or item co-occurrences. Message passing aggregates information from a node's neighbors, yielding embeddings that capture relatedness (e.g., items often bought together).

2.2.2 MITGNN (Base Model)

The Multi-Intent Translation GNN treats a basket not as a single vector but as a mixture of K latent "intents." User, item, and basket embeddings are translated into intent-specific vectors, which are then attended/aggregated to score whether a candidate item fits the basket. This multi-intent view captures heterogeneous trips (e.g., groceries + household + beverages) and provides strong within-basket recommendation signals.

2.2.3 MITGNN + GRNN

To model sequence effects across shopping sessions, we feed per-basket intent representations from MITGNN into a recurrent layer (e.g., LSTM/GRU). The RNN summarizes the ordered history of baskets, enabling next-basket prediction by capturing evolving preferences and periodic purchase patterns.

2.2.4 MITGNN + TGN

To handle irregular, time-stamped interactions, we augment MITGNN with a Temporal Graph Network. TGN maintains node memories and performs time-aware message passing, updating user/item states as events occur. This preserves MITGNN's multi-intent structure while explicitly modeling temporal dynamics in evolving user–item graphs.

2.3 Model Evaluation

To assess the quality of recommendations, we adopt four widely used top-K ranking metrics: Recall@K, Precision@K, Hit Ratio@K, and Normalized Discounted Cumulative Gain (NDCG@K). These metrics capture different aspects of recommendation performance, from coverage to accuracy and ranking quality. In our implementation, metrics are computed on both training and test sets, allowing us to monitor not only generalization ability but also model fitting during training.

Recall@K Recall@K measures the proportion of relevant items that appear among the top K recommendations. Formally, for a given user u with ground-truth item set G_u and predicted top-K list P_u^K:

$$\text{Recall@K}(u) = \frac{|G_u \cap P_u^K|}{|G_u|}.$$

This metric emphasizes coverage, i.e., how many of the items a user actually chose are successfully retrieved.

Precision@K Precision@K evaluates the accuracy of the top-K recommendations by calculating the fraction of recommended items that are relevant:

$$\text{Precision@K}(u) = \frac{|G_u \cap P_u^K|}{K}.$$

This metric penalizes irrelevant recommendations, reflecting the proportion of correct items among those suggested.

Hit Ratio@K Hit Ratio@K is a binary measure that indicates whether at least one of the ground-truth items appears in the top-K recommendations:

$$\text{Hit@K}(u) = \begin{cases} 1, & \text{if } G_u \cap P_u^K \neq \emptyset, \\ 0, & \text{otherwise.} \end{cases}$$

Averaged across all users, Hit Ratio@K reflects the proportion of users for whom the system provides at least one useful recommendation.

Normalized Discounted Cumulative Gain (NDCG@K) NDCG@K evaluates both the presence and the ranking position of relevant items within the top-K list. It uses a logarithmic discount factor to penalize relevant items that appear lower in the ranking. For user u, the Discounted Cumulative Gain (DCG) is defined as:

$$\text{DCG@}K(u) = \sum_{i=1}^{K} \frac{\mathbb{I}\{\, p_i \in G_u \,\}}{\log_2(i+1)}, \qquad \text{NDCG@}K(u) = \frac{\text{DCG@}K(u)}{\text{IDCG@}K(u)}.$$

where p_i denotes the i-th recommended item. NDCG is obtained by normalizing DCG with the ideal DCG (IDCG), i.e.,

$$\text{NDCG@K}(u) = \frac{\text{DCG@K}(u)}{\text{IDCG@K}(u)}.$$

Together, these four metrics provide a comprehensive evaluation framework, capturing coverage (Recall), accuracy (Precision), user-level success (Hit Ratio), and ranking quality (NDCG).

2.4 Data Preparation and Transformation

2.4.1 Raw Data Structure

We used two datasets in our experiments: a publicly available Groceries dataset and a larger Multimodal dataset.

Groceries Dataset The Groceries dataset is a transaction-level file in which each row records a purchase with user ID, date, and item description. Purchases from the same user on the same date are grouped into baskets. The dataset is relatively small but provides interpretable item names for embedding and evaluation.

Multimodal Dataset The Multimodal dataset is richer and larger, containing customer IDs, transaction IDs, timestamps, item IDs, and sales information (quantities and amounts). It enables us to infer price information and to test scalability of models under more realistic retail conditions.

2.4.2 Data Processing

For both datasets, we followed standard preprocessing steps:

- Convert user IDs and item IDs (or item descriptions) into numerical indices.
- Aggregate transactions into baskets, with one basket representing all items purchased by a user in a single shopping trip.
- Apply basic restrictions for computational efficiency: baskets with more than 30 items were removed, and only users with at least 5 valid baskets were retained.

Compared to prior work, we simplified the focus on mapping details. While textual and price-based mappings were constructed for embeddings, the specific procedures are secondary to our main contributions and are therefore only briefly noted here.

2.4.3 Train-Test Dataset Construction

We considered both within-basket and next-basket prediction tasks.

Within-Basket Prediction Each basket was split into observed and held-out items. For Groceries we applied an 80-20 split, and for Multimodal we applied a 50-50 split. The goal is to predict the missing items given the partially observed basket.

Next-Basket Prediction Following the standard approach, for each user the last basket was held out as the test instance, with all previous baskets used for training. This setup captures temporal evolution and simulates future purchase prediction.

2.5 Summary of Contributions

Building on prior work, we introduce methodological and implementation improvements that enhance both the accuracy and usability of basket prediction. First, we migrated the codebase from TensorFlow 1 to PyTorch 2, modernizing the stack and resolving interface inconsistencies between the MITGNN backbone and its GRNN/TGN extensions, and supporting multi-GPU training. Second, we reduced repeated recommendations by refining the inference pipeline, yielding more diverse and useful outputs. Third, we enhanced temporal features passing by improving time encoding and introducing a sliding-window mechanism to better capture sequential dynamics. Finally, we broadened configuration and training options, improving robustness across datasets and experimental settings.

Details of these improvements—their design and empirical impact—are provided in the following section.

3 PyTorch Framework

3.1 Motivation

The legacy TensorFlow v1 system maintained three separate pipelines (MITGNN, GRNN, TGN), each duplicating data loading, configuration, training,

and evaluation logic. This fragmentation increased maintenance cost, introduced configuration drift, and hindered extensibility. Moreover, TensorFlow v1 is now legacy, making long-term maintenance and onboarding costly.

3.2 Unified PyTorch Framework

We consolidate all three models into a single, production-ready PyTorch pipeline driven by compact, hierarchical YAML configuration. A unified entry point orchestrates data, model, training, and evaluation with consistent logging, checkpointing, and metrics, while enabling variant-specific behavior through well-defined extension points.

3.3 Core Components

- **Configuration (Hierarchical YAML)**: Deep-merge inheritance with override precedence eliminates duplication. Base files (`base/common.yaml`, `base/mitgnn_base.yaml`, `base/grnn_base.yaml`, `base/tgn_base.yaml`) define defaults; experiment files are 6–15 lines. Validation uses Pydantic; circular inheritance is detected; paths resolve relative to `configs/`.

- **Data Layer (Variant-Specific Modules)**: `MITGNNDataModule`, `GRNNDataModule`, `TGNDataModule` prepare datasets/loaders for within-basket and next-basket tasks. Expected inputs: `train_u2b.txt`, `train_b2i.txt`, `test_b2i.txt`, and optional `basket_timestamps.txt` (TGN). Loader knobs (batch size, workers, pin memory) are set in config.

- **Models (Unified Interface)**: `MITGNNModel`, `GRNNModel`, `TGNModel` implement a common interface, including `get_all_item_scores(user_ids, basket_ids)` for full-ranking evaluation. Capabilities: multi-intent GNN layers, sequential RNN components (LSTM/GRU), temporal encodings (cosine/sinusoidal/learned), optional price features, and configurable prediction heads (`mlp` or `dot_product`).

- **Trainer (UnifiedTrainer)**: Single training loop with Adam, pluggable LR schedulers (step, multistep, exponential, cosine, warmups, plateau), early stopping, gradient clipping, checkpointing (best/final), and TensorBoard. Supports BPR or BCE-with-logits losses and negative sampling when needed. Multi-GPU via `DataParallel` with automatic device selection.

- **Evaluation (UnifiedEvaluator)**: Full-ranking metrics over the entire item set with optional filtering of training items for fair comparison. Metrics include Recall@K, NDCG@K, Hit Ratio@K, Precision@K, plus AUC/AP and coverage/novelty where configured. A `PredictionAnalyzer` provides visibility into per-user predictions.

- **Utilities**: Structured logging (console/file/TensorBoard), reproducibility (seed + deterministic modes), scheduler factory, and consistent output directories for checkpoints and logs.

3.4 Usage

Basic runs (all configuration in YAML):

```
python3 pytorch_pipeline/run.py --config configs/mitgnn_within.yaml
python3 pytorch_pipeline/run.py --config configs/grnn_next.yaml
```

3.5 YAML File Setting Example

Common defaults in `common.yaml`. We place global defaults in `common.yaml`, which acts as the foundation for all model variants. The settings are structured into four main blocks:

- **training**: learning rate, weight decay, number of epochs, BPR loss type, various dropouts, early stopping criteria (patience, monitored metric, tolerance), checkpointing frequency, and whether to compute ranking metrics on the training set. A flexible learning rate scheduler is also supported, with options including warmup cosine, multistep, cosine restart, plateau, and polynomial.

- **evaluation**: unified metrics (recall, precision, ndcg, hit ratio), the list of K values to report, test batch size, evaluation frequency, and a `prediction_analysis` block that can save JSON/CSV outputs, raw scores, and per-example losses for visibility and diagnostic analysis.

- **system**: reproducibility and runtime settings such as device auto-selection (CPU/GPU), random seed, deterministic execution, logging, TensorBoard output, cache directory, and multi-GPU configuration (auto-detection, strategy, device IDs).

- **data**: dataloader settings (batch size, workers, pinned memory) and default sliding-window options (window length, stride, mode, leakage prevention, and aggregate features).

Referencing the base with `mitgnn_base`. To avoid redundancy, experiment YAMLs reference the base configuration through `base_config`. For example, `mitgnn_base.yaml` inherits all defaults from `common.yaml` and specifies only the MITGNN backbone (variant, embedding size, layer sizes, adjacency type, intent module, prediction method, regularization weight) and dataset path.

Override rule. Any setting in the experiment file overrides the corresponding entry in the `base_config`. Thus, we inherit stable defaults from `common.yaml` and selectively adjust hyperparameters or data paths as needed, ensuring consistency while keeping experiments concise.

Minimal example (`mitgnn_base.yaml`).

```
# mitgnn_base.yaml
base_config: "base/common.yaml"   # inherit defaults

model:
  variant: "mitgnn"
  embed_size: 64
  layer_sizes: [64, 64, 64]
  adj_type: "norm"
  alg_type: "intent_conv"
  num_intent: 5
  sigma: 0.9
  prediction_method: "dot_product"
  use_explicit_regularization: true
  regularization_weight: 0.001

data:
  data_path: "/path/to/mitgnn/data"

# Optional overrides for this run
training:
```

```
  lr: 0.0005
  epochs: 150
  scheduler:
    type: "polynomial"
    warmup_epochs: 5
    eta_min: 0.0
    power: 1.0

evaluation:
  top_k: [10, 20]
  prediction_analysis:
    enabled: true
    save_history: true
```

This example shows how `common.yaml` provides a stable baseline, while `mitgnn_base.yaml` remains a short and readable diff. Any same-named field here will replace the inherited default, enabling flexible yet consistent experimentation.

Extending beyond MITGNN. In addition to serving as a concrete experiment configuration, `mitgnn_base.yaml` itself can also act as a new `base_config`. In our framework, we defined similar base files for GRNN and TGN, each of which inherits from `common.yaml` and specifies the corresponding backbone architecture. When running experiments, users can therefore set the base to `mitgnn_base`, `grnn_base`, or `tgn_base`, and then override only the model-specific hyperparameters or data paths. This layered design keeps experiment files short, readable, and highly reusable while ensuring consistency across model families.

4 Advanced Modules and Model Improvement

4.1 Within-basket and Next-basket Prediction

4.1.1 Within-basket Prediction

Within-basket prediction aims to infer the remaining items that a user may add to an ongoing basket. In this setup, the basket is only partially observed: some items have already been placed in the cart, while others are still missing.

The model's task is to recommend which additional items are most likely to appear before checkout.
Example. Suppose a customer has already added {*bread, butter*} to the basket. A good within-basket model should be able to recommend related items such as {*jam, milk*}, reflecting typical co-purchase patterns. This task is particularly useful in online retail, where the system can make real-time suggestions during the shopping process to increase basket size and improve user experience.

4.1.2 Next-basket Prediction

Next-basket prediction instead focuses on forecasting the contents of a user's *future* shopping trip. Given the entire sequence of a user's past baskets, the goal is to predict what the user is most likely to purchase in their next session. This task emphasizes sequential and temporal modeling, since user preferences and purchase needs evolve over time.
Example. Consider a user whose purchase history consists of:

- Basket 1: {*bread, beverage, vegetables*}
- Basket 2: {*bread, diary, beverage*}

When predicting Basket 3, the model might suggest {*bread, vegetables, beverage*}, capturing not only co-purchase tendencies but also the temporal evolution of cooking-related shopping behavior. This type of prediction is particularly valuable for customer retention and targeted marketing, since it allows retailers to anticipate demand across multiple visits.

4.2 Improvement of time embedding setting and sliding window adoption

Legacy limitation. The prior TensorFlow v1 pipelines treated timestamps as raw floating numbers, which made models sensitive to time scale and distribution shifts, and offered no principled temporal inductive bias.

Time encoding (configurable, pluggable). We introduce a structured temporal module with (i) a normalization hook and (ii) flexible time encodings:

- **Normalization** (`temporal_normalization`): default `raw`. The hook is designed for extensibility (e.g., min–max or z-score) without changing callers.
- **Encodings** (`temporal_encoding`): `cosine`, `sinusoidal`, or `learned`. For sinusoidal, with embedding dim d and base τ,

$$\mathrm{PE}_{2k}(t) = \sin\Big(t/\tau^{2k/d}\Big), \quad \mathrm{PE}_{2k+1}(t) = \cos\Big(t/\tau^{2k/d}\Big), \quad k = 0, \ldots, \lfloor d/2 \rfloor - 1.$$

- **Weighting** (`temporal_weight`): scalar to modulate temporal features in the model head.

These changes yield stable scale handling, better periodicity capture, and cleaner ablations across datasets.

Sliding-window adoption (sequence summarization). We add a configurable sliding-window mechanism that summarizes recent user behavior over baskets or days:

$$\mathcal{W}_u^{(t)} = \{\, \text{events in } [t - L + 1,\, t] \,\} \;\text{ with stride } s,$$

optionally including the current basket. Each window is featurized with efficient aggregates:

- Counts and diversity: number of baskets, number of unique items, average items per basket.
- Temporal gaps: time since last event, median inter-event gap.
- *Decayed* statistics with factor $\alpha \in (0, 1]$: e.g., $S_\alpha = \sum_i \alpha^{\Delta t_i}\, \phi(x_i)$.

This preserves short-term intent while remaining robust to noise, and unifies within-/next-basket setups.

Configuration (compact examples).

```
model:
  tgn:
    temporal_dim: 2
    temporal_encoding: "cosine"     # or "sinusoidal" | "learned"
```

```
    temporal_normalization: "raw"   # hook, default raw
    temporal_weight: 1.0

data:
  sliding_window:
    enabled: true
    length: 5
    stride: 1
    mode: "baskets"                 # or "days"
    include_current_basket: false
    aggregates:
      enabled: true
      features:
        - num_baskets
        - num_unique_items
        - avg_items_per_basket
        - time_since_last
        - median_time_gap
      decay_alpha: 0.8
      max_windows_per_user: 10
```

4.3 Extended Model Configurations

Early Stopping To mitigate overfitting and reduce unnecessary computation, we adopt patience-based early stopping on validation metrics. Let m_t be the monitored validation metric at epoch t (e.g., Recall@K or NDCG@K). Given tolerance $\delta \geq 0$ and patience $p \in \mathbb{N}$, training stops at epoch T if there is no improvement beyond δ for p consecutive epochs:

$$\max_{t' \in \{T-p,\ldots,T\}} m_{t'} \leq m^\star + \delta, \quad m^\star = \max_{t<T-p} m_t \ \text{(for "maximize" metrics)}.$$

When triggered, the best-performing checkpoint is retained, ensuring stronger generalization and avoiding wasted computation.

Visibility Enhancement Beyond improving raw performance, it is equally important to make model behavior transparent and interpretable. To this end, we developed a dedicated `prediction_analysis` module that provides

multiple forms of logging, visualization, and output saving. These features allow researchers and practitioners to better understand how the model learns, where it succeeds, and where it fails.

Concretely, the module offers the following functions:

- **Flexible configuration.** Users can enable or disable the analyzer, control the number of examples logged per mode (`max_examples`), and specify which top-K thresholds to display (`top_k_show`). Additional options include saving outputs to files (`save_to_file`, `save_history`), mapping item IDs to names for interpretability (`show_item_names`), and exporting raw scores or per-example losses for detailed inspection.

- **Per-mode JSON dumps.** For each dataset split (`train`, `val`, `test`), the analyzer generates JSON files (e.g., `predictions_{mode}_{tag}.json`). These contain the input baskets, top-K predictions, ground-truth comparisons, and optional raw scores. They serve as a traceable record of the model's recommendation behavior at the individual-user level.

- **Epoch history tracking.** When `save_history` is enabled, the system produces a CSV log (`epoch_history_{run_tag}.csv`) that records both training and evaluation metrics at every epoch. This file can be used to visualize learning curves, identify overfitting, and compare different configurations systematically.

These outputs provide more than just diagnostic information. They make it possible to answer research questions such as: *Does the model improve predictions for certain users over time? Which items are consistently mispredicted? How do top-K hit rates evolve with training epochs?* In practice, this visibility allows us to detect issues such as repeated recommendations, quantify the effects of architectural changes, and generate richer qualitative analyses that complement numerical metrics.
Example output files are summarized in Table 1.

Learning Rate Schedulers We integrated four complementary learning rate schedulers to improve training stability and convergence. Each of them follows a different strategy for adjusting the step size of gradient updates, balancing fast learning in the early stages with stability and generalization later on. Below we describe their intuition and corresponding update rules.

File	Content
`predictions_train_exp1.json`	Top-K predictions and hits for training examples
`predictions_val_exp1.json`	Validation set predictions with raw scores (if enabled)
`predictions_test_exp1.json`	Test set predictions with per-user history
`epoch_history_exp1.csv`	Metrics and loss per epoch for train/val/test

Table 1: Illustration of visibility outputs produced by the `prediction_analysis` module.

(1) LinearWarmupLR [1] This scheduler linearly increases the learning rate from zero to the base value ℓ_0 during the first W epochs, and then keeps it constant. The warm-up avoids unstable updates at the very beginning of training, especially when using large batch sizes or complex models.

$$\ell(e) = \begin{cases} \ell_0 \cdot \dfrac{e+1}{W}, & e < W, \\ \ell_0, & e \geq W. \end{cases}$$

(2) WarmupCosineAnnealingLR [2] This strategy combines linear warm-up with a smooth cosine decay. After the warm-up phase, the learning rate gradually decreases from ℓ_0 to a minimum value $\eta_{\min}$ following a half-cosine curve. The cosine schedule provides a gentle reduction, often helping the model converge to flatter minima and improving generalization.

$$\ell(e) = \begin{cases} \ell_0 \cdot \dfrac{e+1}{W}, & e < W, \\ \eta_{\min} + (\ell_0 - \eta_{\min}) \cdot \dfrac{1 + \cos\left(\pi \cdot \frac{e-W}{E-W}\right)}{2}, & e \geq W. \end{cases}$$

(3) CosineWarmupRestartLR (SGDR-style) [4] This scheduler extends the cosine annealing approach by periodically "restarting" the learning rate to ℓ_0, then decaying again according to the cosine rule. The restart intervals grow geometrically (with initial cycle length T_0 and multiplier T_{mult}). This cyclic behavior allows the optimizer to escape sharp local minima and explore new regions of the loss surface, often improving robustness on difficult

datasets.

$$\ell(e) = \begin{cases} \ell_0 \cdot \dfrac{e+1}{W}, & e < W, \\ \eta_{\min} + (\ell_0 - \eta_{\min}) \cdot \dfrac{1 + \cos\left(\pi \cdot \frac{t}{T_i}\right)}{2}, & e \geq W, \end{cases}$$

where t is the position within cycle i of length T_i.

(4) PolynomialLR Finally, the polynomial scheduler decays the learning rate from ℓ_0 to $\eta_{\min}$ following a polynomial curve of degree p. A higher p leads to a sharper drop near the end of training, while a smaller p yields a smoother, slower decay. This flexibility makes the polynomial schedule useful when one wants explicit control over how aggressively the learning rate is reduced.

$$\ell(e) = \begin{cases} \ell_0 \cdot \dfrac{e+1}{W}, & e < W, \\ \eta_{\min} + (\ell_0 - \eta_{\min}) \cdot \left(1 - \frac{e-W}{E-W}\right)^p, & e \geq W. \end{cases}$$

5 Experiment

5.1 Different Split Test

5.1.1 Population Split

To study the effect of dataset scale, we introduced a population-based split. Specifically, we partitioned the user set into five different sizes (20%, 40%, 60%, 80%, and 100%) to create training and testing subsets. Since the split is based on users, consistency is maintained between training and test sets—each subset forms a self-contained population. This design enables us to empirically examine scale-law hypotheses by observing how performance changes as the number of users grows.

5.1.2 Different Split Methods

In addition to population scaling, we also examined alternative strategies for next-basket construction. While the previous team mainly relied on holding out the last basket per user, we experimented with:

1. **Random basket split:** selecting a random basket as the test set.

2. **User-based split:** holding out an entire subset of users for testing.

However, our experiments showed that the original method—using the last basket as the test set—remains the most consistent and meaningful, as it respects the natural temporal sequence of shopping behavior. We therefore adopted this strategy as our primary evaluation setting.

5.2 Experiment Setup

All experiments were conducted on the Groceries dataset using the following training and model configuration. The model employed an embedding size of 256 with three hidden layers of size 256 each. The training setup used a learning rate of 3e-5, a maximum of 200 epochs, weight decay of 1×10^{-4}, and dropout set to 0.2:

```
model:
  embed_size: 256
  layer_sizes: [256, 256, 256]

training:
  lr: 0.00003
  epochs: 200
  weight_decay: 1e-4
  dropout: 0.2
```

We tested several model-task combinations, including MITGNN and TGN variants, on both within-basket and next-basket prediction tasks. For TGN, we additionally compared settings with and without sliding window.

5.3 Experiment Results

The validation results for each setting are summarized in Table **??**. Metrics include Recall, NDCG, Hit Ratio, and Precision.

	Recall	NDCG	Hit Ratio	Precision
@10	44.22%	31.06%	44.22%	4.42%
@20	47.53%	33.60%	47.53%	2.38%

Table 2: Validation results for **Groceries MITGNN (within-basket)**.

	Recall	NDCG	Hit Ratio	Precision
@10	32.69%	24.80%	64.50%	8.94%
@20	46.77%	29.33%	78.90%	6.50%

Table 3: Validation results for **Groceries MITGNN (next-basket)**.

5.3.1 Groceries Data

	Recall	NDCG	Hit Ratio	Precision
@10	6.01%	2.35%	16.15%	1.65%
@20	27.91%	9.32%	57.54%	3.81%

Table 5: Validation results for **Groceries TGN (next-basket, with sliding window)**.

6 Discussion

6.1 Result Interpretation

The experimental results reveal several notable patterns regarding model performance across different tasks and configurations.

MITGNN Performance. On the Groceries dataset, MITGNN achieves consistently strong results in both within-basket and next-basket prediction. For within-basket prediction, Recall@10 reaches 44.22% with NDCG@10 at 31.06%, indicating that the model is effective at capturing co-purchase relationships inside a single basket. For next-basket prediction, performance remains competitive (Recall@10 = 32.69%, NDCG@10 = 24.80%), and the Hit Ratio is substantially higher (64.50%), suggesting that MITGNN generalizes well to sequential forecasting. The similarity between within- and next-basket outcomes highlights the robustness of MITGNN's intent-based embedding framework.

	Recall	NDCG	Hit Ratio	Precision
@10	23.53%	13.27%	23.53%	2.35%
@20	40.03%	17.55%	40.03%	2.00%

Table 4: Validation results for **Groceries TGN (within-basket)**.

TGN Performance. In contrast, TGN shows weaker results. For within-basket prediction, Recall@10 drops to 23.53% and NDCG@10 to 13.27%, suggesting that its temporal-memory design does not adapt well when the task only requires modeling intra-basket relationships. In next-basket prediction with sliding window, results further decline (Recall@10 = 6.01%, NDCG@10 = 2.35%), though Recall@20 improves moderately to 27.91%. These findings suggest that TGN struggles with the relatively small size and sparse structure of the grocery dataset, particularly under sliding-window segmentation.

Precision Considerations. Across all models, Precision values are relatively low compared to Recall and Hit Ratio. This can be partly explained by the nature of the grocery baskets: most baskets contain only 3–4 items, as shown in our earlier data analysis. Because Precision@10 and Precision@20 evaluate the proportion of relevant items within a fixed-length recommendation list, even accurate predictions appear diluted when users do not typically purchase 10 or more items at once. Thus, low Precision scores do not necessarily indicate poor recommendation quality but rather reflect the mismatch between basket size distribution and the evaluation metric.

Overall Insights. Taken together, these results indicate that intent-based architectures such as MITGNN are well suited for both within- and next-basket tasks in grocery data, capturing both item co-occurrence and sequential dynamics with reasonable effectiveness. Temporal memory approaches like TGN, although promising in principle, may require larger and denser datasets to fully demonstrate their advantages. Furthermore, the limited gains from sliding-window processing highlight that preprocessing choices must be aligned with dataset characteristics in order to be effective.

Comparison with Prior Work. Relative to the prior team's results on the Groceries dataset, our models achieve substantial improvements, espe-

cially for MITGNN. The previous baseline reported Recall@10 of 25.78%, NDCG@10 of 13.04%, and Precision@10 of 2.58%. In our experiments, MITGNN reached 32.69% Recall@10, 24.80% NDCG@10, and 8.94% Precision@10. These correspond to an increase of nearly 7 percentage points in recall, more than 11 percentage points in NDCG, and over a threefold improvement in precision. Such gains demonstrate the benefits of our updated framework, including the PyTorch migration, refined model connections, and enhanced configurations. Although TGN did not perform as well as MITGNN in absolute terms, its results under our pipeline remain consistent, highlighting the robustness of our training and evaluation setup compared to the earlier implementation.

6.2 Future Work

6.2.1 Model Design Part

Several research directions remain open in terms of model design and methodological advances:

- **Scaling Up.** Extending experiments to larger multimodal datasets will allow us to evaluate population-size effects and test the scalability of our models beyond the grocery domain. Such scaling studies are essential for verifying whether observed trends follow predictable scale laws.

- **Neural Architecture Search.** Automated search methods can be used to identify optimal model architectures for different data sets. Rather than relying on manual tuning, NAS may reveal non-obvious combinations of layers, hidden dimensions, or attention mechanisms that better capture purchasing behavior.

- **Task Optimization.** A systematic comparison between within-basket and next-basket setups is needed to determine which model configurations perform best under different task formulations. This will help clarify trade-offs between short-term basket completion and long-term purchase forecasting.

- **Model Selection.** Further work should establish clearer guidelines for when to deploy MITGNN, GRNN, or TGN. Each architecture has

unique strengths, including multi-intent modeling, sequential dependency capture, or temporal memory, and understanding their trade-offs will improve model interpretability and applicability.

- **New Embedding Injection.** Future models should be designed to seamlessly incorporate new information sources (e.g., product descriptions, category hierarchies, or price attributes in multimodal data). Flexible embedding injection will improve adaptability to heterogeneous datasets and evolving product catalogs.

6.2.2 Practice Part

On the practical side, several strategies can make the models more robust and usable in real-world retail scenarios:

Pretraining and Fine-tuning We envision a two-stage process. First, the model is pretrained on a small set of representative stores, where it learns general purchasing patterns across customers and items. The resulting checkpoint captures common structures in user-item interactions. Later, fine-tuning is applied on a new store not included in pretraining: only a few epochs are required, since most parameters already encode general behaviors. This allows rapid adaptation to store-specific conditions, such as regional preferences or unique product bundles, while keeping the computational cost low.

Global Item Embeddings Item embeddings are maintained in a global dictionary shared across all stores. When a new SKU appears, we assign it a global ID and initialize its embedding (randomly or using side information). During fine-tuning and subsequent updates, this embedding is quickly refined. The global dictionary ensures cross-store consistency and avoids retraining embeddings from scratch for every store.

Store-specific User and Basket Embeddings Unlike items, user and basket embeddings can remain store-specific. For a new user, a fresh embedding is created during fine-tuning and updated with that user's basket history. In cold-start situations with very few transactions, personalization is limited but improves as more data is collected. Basket embeddings follow

a similar principle: they are generated dynamically from the items present in each basket and are iteratively refined as the user interacts with the system.

7 Conclusion

In this work, we developed a unified PyTorch framework for basket prediction, extending and refining prior implementations of MITGNN, GRNN, and TGN. By migrating from TensorFlow v1 to PyTorch, we resolved compatibility issues across models and introduced a consistent modular pipeline that improves both usability and extensibility. Additional methodological advances, including refined time embeddings, sliding window mechanisms, early stopping, and flexible learning rate schedulers, further enhanced the stability and interpretability of training. We also added a dedicated prediction analysis module that improved visibility, enabling per-user history tracking and richer diagnostic output.

Empirical evaluation on the Groceries dataset demonstrates clear performance improvements compared to prior work. In particular, MITGNN achieved higher recall, NDCG, and precision across both within-basket and next-basket tasks, validating the effectiveness of our improvements. While TGN exhibited weaker performance on this relatively small and sparse dataset, our results remain consistent and robust under the unified pipeline, laying a foundation for more advanced temporal modeling in larger-scale settings.

Overall, our contributions provide both a stronger technical foundation and measurable gains in predictive accuracy. By consolidating models into a reusable PyTorch framework and extending their configuration and analysis capabilities, we not only improve research reproducibility but also move closer to real-world applicability. Future work will explore neural architecture search, richer integration of embedded components, and practical adaptation strategies such as pre-training and fine-tuning across multiple stores, with the aim of making basket prediction models more adaptive, scalable, and impactful in modern retail environments.

References

[1] Priya Goyal, Piotr Dollar, Ross Girshick, Pieter Noordhuis, Lukasz Wesolowski, Aapo Kyrola, Andrew Tulloch, Yangqing Jia, and Kaim-

ing He. Accurate, large minibatch sgd: Training imagenet in 1 hour. *arXiv preprint arXiv:1706.02677*, 2017.

[2] Tong He, Zhi Zhang, Hang Zhang, Zhongyue Zhang, Junyuan Xie, and Mu Li. Bag of tricks for image classification with convolutional neural networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 558–567, 2019.

[3] Zhiwei Liu, Xiaohan Li, Ziwei Fan, Stephen D. Guo, Kannan Achan, and Philip S. Yu. Basket recommendation with multi-intent translation graph neural network. In *2020 IEEE International Conference on Big Data (Big Data)*, pages 728–737. IEEE, 2020.

[4] Ilya Loshchilov and Frank Hutter. Sgdr: Stochastic gradient descent with warm restarts. In *International Conference on Learning Representations (ICLR)*, 2017. arXiv:1608.03983.

[5] Emanuele Rossi, Ben Chamberlain, Fabrizio Frasca, Davide Eynard, Federico Monti, and Michael M. Bronstein. Temporal graph networks for deep learning on dynamic graphs. *arXiv preprint arXiv:2006.10637*, 2020. doi: 10.48550/arXiv.2006.10637.

[6] Luana Ruiz, Fernando Gama, and Alejandro Ribeiro. Gated graph recurrent neural networks. *IEEE Transactions on Signal Processing*, 68: 6303–6318, 2020. doi: 10.1109/TSP.2020.3033962.